SOFTWARE DESIGN LAB • SECOND YEAR ENGINEERING

Payment Processing System

A Configurable Design using the Strategy Pattern

Select and switch payment methods at run time — without changing the core processing logic.

Strategy Pattern **Credit Card** **PayPal** **UPI** **Net Banking**

Practical Lab Manual & Presentation

PROBLEM STATEMENT

What We Are Building

Write a program to implement a Configurable Payment Processing System Using Strategy Pattern.

G

Goal

Support many payment methods and let the caller pick / switch one at run time.

H

How

A common PaymentStrategy interface; each method is its own interchangeable class.

O

Output

A PaymentProcessor (Context) that delegates to the currently configured strategy.

LAB OBJECTIVES

Aim & Objectives

AIM

To design and implement a configurable payment processing system in Python using the Strategy design pattern, so the payment method can be selected and switched at run time without changing the core processing logic.

OBJECTIVES

- Understand the Strategy pattern's intent, structure, and use cases.
- Design a common interface multiple interchangeable algorithms can implement.
- Build a Context class that delegates work instead of using if/elif chains.
- Make the system configurable — register and select methods by name at run time.
- Combine Strategy with a decorator (logging) and classmethods (factory/registry).

BEFORE YOU BEGIN

Prerequisites

Language basics

Python 3.x syntax, functions, control statements

OOP basics

Classes, objects, inheritance, method overriding

Abstraction

abc module: ABC and @abstractmethod

Design pattern basics

Behavioural vs. structural vs. creational patterns

classmethod

Factory-style construction, shared class data

Decorators

Basic @decorator syntax (used for logging here)

Setup

Python 3.8+, an IDE (VS Code / PyCharm), terminal access

Why the Strategy Pattern?

Strategy is a behavioural pattern: it defines a family of interchangeable algorithms, puts each in its own class, and lets the algorithm vary independently of the client that uses it.

- No if/elif chain for 'which payment method is this?'
- New methods are added as NEW classes — Context stays unchanged
- Each method is independently testable in isolation
- Respects the Open/Closed Principle (open to extension, closed to modification)

```
class PaymentStrategy(ABC):
    name = "Generic Payment"

    @abstractmethod
    def validate(self): ...

    @abstractmethod
    def pay(self, amount): ...

class UPIPayment(PaymentStrategy):
    name = "UPI"
    def __init__(self, upi_id):
        self.upi_id = upi_id
    def validate(self):
        return "@" in self.upi_id
    def pay(self, amount): ...
```

The Context: PaymentProcessor

The Context holds a reference to the CURRENT strategy and delegates all real work to it. It never contains payment-method-specific logic.

- strategy : PaymentStrategy → the currently active strategy
- set_strategy(strategy) → swap the strategy at run time
- process_payment(amount) → delegates to strategy.pay(amount)
- _registry : dict → class attribute mapping key → strategy class

```
class PaymentProcessor:
    _registry = {}

    def set_strategy(self, strategy):
        self.strategy = strategy

    @log_transaction
    def process_payment(self, amount):
        return self.strategy.pay(amount)

    @classmethod
    def register_strategy(cls, key, s_cls):
        cls._registry[key] = s_cls

    @classmethod
    def create(cls, key, **kwargs):
        s_cls = cls._registry[key]
        return cls(s_cls(**kwargs))
```

ELABORATION · PART 3

What Makes It 'Configurable'?

Two mechanisms let the system change behaviour at run time without editing PaymentProcessor's source code.

set_strategy()

```
processor.set_strategy(x)
```

register_strategy()

```
cls._registry[key] = s_cls
```

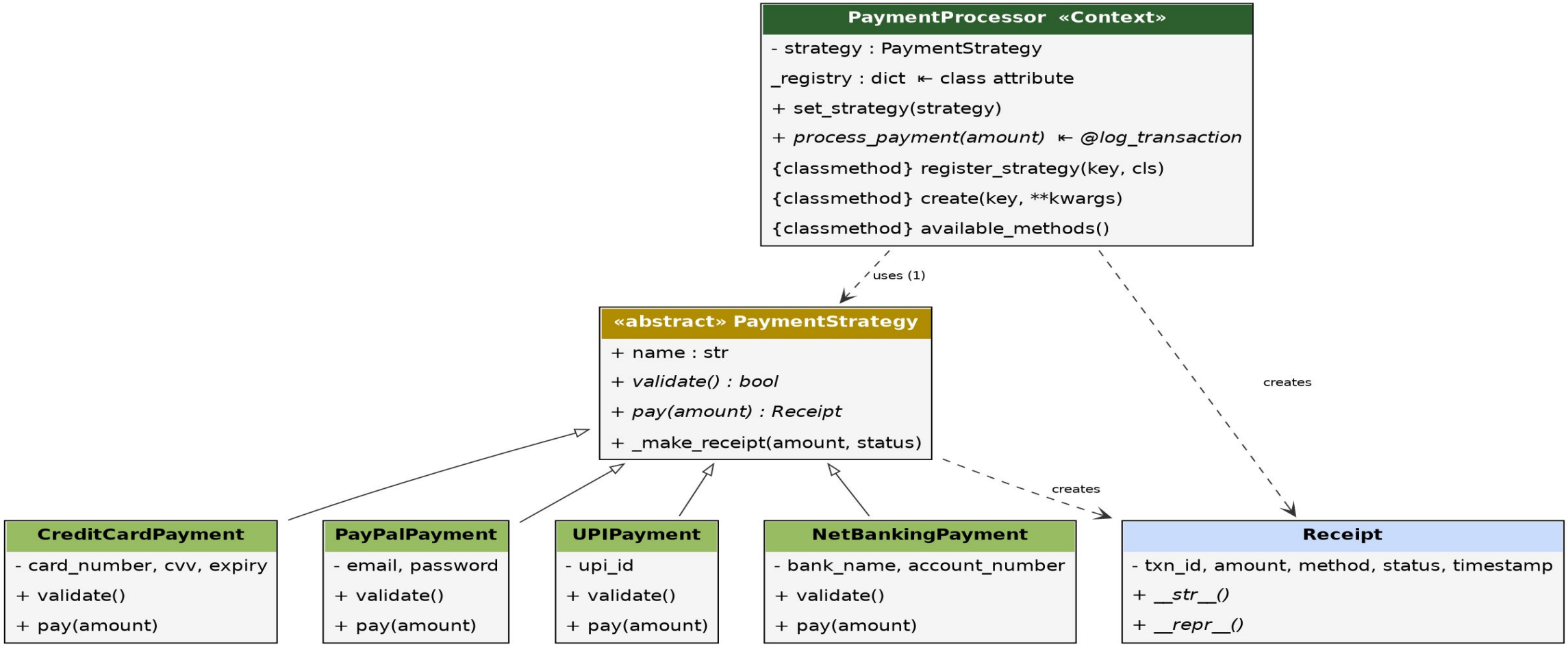
create()

```
PaymentProcessor.create("upi", ...)
```

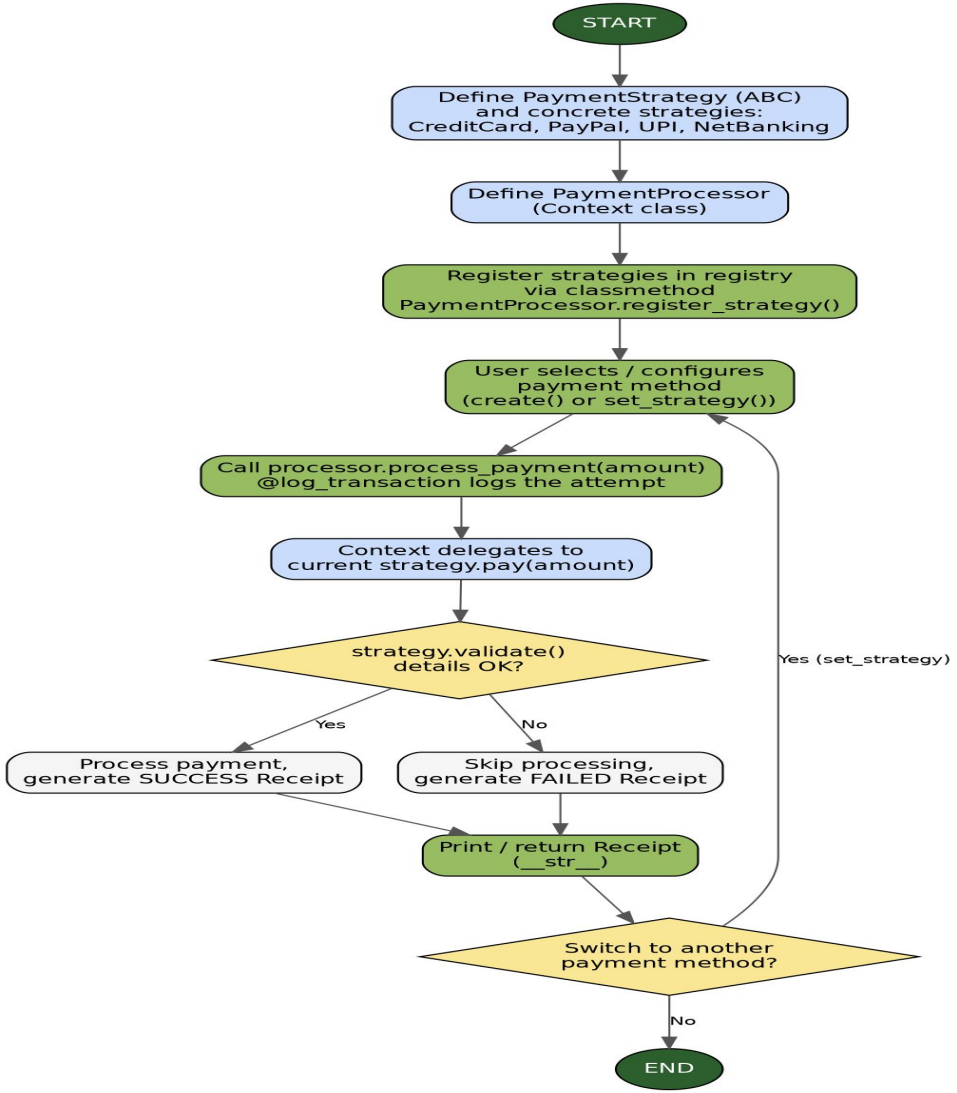
available_methods()

```
list(cls._registry.keys())
```

Class Diagram



Program Flowchart



Step-by-Step: Steps 1–9

1. Create payment_processor.py; import ABC, abstractmethod, functools, uuid, datetime.
2. Define the Receipt class (txn_id, amount, method, status, timestamp) with __str__.
3. Define abstract PaymentStrategy(ABC) with validate() and pay() as @abstractmethod.
4. Implement CreditCardPayment with card_number, cvv, expiry, validate(), pay().
5. Implement PayPalPayment with email, password, validate(), pay().
6. Implement UPIPayment with upi_id, validate(), pay().
7. Implement NetBankingPayment with bank_name, account_number, validate(), pay().
8. Define the log_transaction decorator to log before/after each payment.
9. Define PaymentProcessor with strategy, set_strategy(), and process_payment().

Step-by-Step: Steps 10–18

10. Add _registry = {} as a class attribute on PaymentProcessor.
11. Implement classmethods register_strategy(), available_methods(), create().
12. In the driver code, register all four strategies with register_strategy().
13. Build a processor via PaymentProcessor.create("upi", upi_id="...").
14. Call process_payment() and print the returned Receipt.
15. Demonstrate set_strategy() switching to Credit Card, then Net Banking.
16. Demonstrate an intentionally invalid strategy to show a FAILED receipt.
17. Run: python payment_processor.py from the terminal.
18. Compare console [LOG]/[CONFIG]/[REGISTRY] output against expected behaviour.

IMPLEMENTATION HIGHLIGHT

Configuring by Name, Not by Code Change

```
PaymentProcessor.register_strategy(
    "upi", UPIPayment)
PaymentProcessor.register_strategy(
    "credit_card", CreditCardPayment)

# build a fully-configured processor
# just by naming a key:
processor = PaymentProcessor.create(
    "upi", upi_id="user@bank")

receipt = processor.process_payment(1500)
```

Because strategies are looked up from a string key stored in _registry, the exact same code that reads a config file or a database value can decide which PaymentStrategy to use — no source-code edits, no redeploy.

Key idea: the Context stays generic; new strategies plug in from OUTSIDE the class.

STUDENT HANDOUT

Strategy vs. Plain if/elif Design

Aspect	if/elif Approach	Strategy Pattern Approach
Adding a new method	Edit the existing class	Add one new class; Context untouched
Testability	Hard to isolate one branch	Each strategy tested independently
Open/Closed Principle	Violated — must modify class	Respected — class stays closed
Run-time switching	Re-check a flag every call	Just call set_strategy() once

Remember: Strategy pattern participants — Strategy (interface), Concrete Strategy, Context, and Client (chooses the strategy).

STUDENT HANDOUT

Viva / Quiz Questions

Q1.
What problem does Strategy solve vs. a long if/elif chain?

Q2.
Why is PaymentStrategy declared using ABC and @abstractmethod?

Q3.
Why is PaymentProcessor called the 'Context'?

Q4.
How does _registry make the system configurable, not hard-coded?

Q5.
Difference between set_strategy() and create()?

Q6.
If you add CryptoPayment, which class(es) must change?

Q7.
What happens if process_payment() runs with no strategy set?

Q8.
How does @log_transaction stay separate from payment logic?

CONCLUSION

One Interface, Many Interchangeable Behaviours

- Strategy pattern → each payment method is its own class behind a common interface.
- Context (PaymentProcessor) → delegates, never contains method-specific logic.
- Registry + classmethods → new methods plug in by registering, not by editing code.
- Together, these fulfil the problem statement: a configurable payment system.